Phase 1 Acceptance Report

Application Health Assessment &
Enterprise Readiness Review

Intelligence Platform | Pre-Phase 2 Gate Review

Date: July 24, 2026
Classification: Internal / Confidential
Version: 1.0

Table of Contents

1. **Current Application Status**
   Build, lint, tsc, and Prisma validation results
2. **@ts-nocheck File Analysis**
   Complete inventory of 42 suppressed files with disposition
3. **User Journey Validation**
   End-to-end enterprise customer onboarding flow
4. **Database Reality Check**
   67 Prisma models: used, unused, and orphan analysis
5. **Functional Testing Status**
   Comprehensive test plan covering all critical paths
6. **UI/UX Assessment**
   Screen inventory and product maturity classification
7. **Phase 2 Plan**
   Objectives, files, risks, and acceptance criteria

1. Current Application Status

This section provides the raw diagnostic output from the four canonical checks: **npm run build**, **npm run lint**, **npx tsc --noEmit**, and **npx prisma validate**. These commands were executed on July 24, 2026 against the current **main** branch in the project workspace at `/home/z/my-project/`.

1.1 Build Status (npm run build)

The production build completes successfully. The build pipeline runs `prisma generate` followed by `next build`. Notably, `ignoreBuildErrors` has been set to `false` in `next.config.ts`, meaning Next.js no longer silently skips TypeScript errors during compilation. However, the build still passes because `noImplicitAny` is set to `false` in `tsconfig.json`, and 42 files use `@ts-nocheck` directives that suppress all type checking within those files. The build produces 120+ API routes and 4 static pages (app, login, signup, sitemap.xml). All pages and API routes compile without fatal errors.

Check	Result	Status
npm run build	Success - 120+ API routes, 4 static pages	PASS
ignoreBuildErrors	false (correctly disabled)	PASS
Prisma generate	Success (v6.19.3)	PASS
Build duration	Completes within 120s	PASS
Static pages	/app, /login, /signup, /sitemap.xml	INFO

1.2 TypeScript Check (npx tsc --noEmit)

The TypeScript compiler reports **0 errors** when invoked with `--noEmit`. This is **not** because the codebase is fully type-safe. The zero-error result is achieved through a combination of three mechanisms: (1) `noImplicitAny: false` in tsconfig.json, which allows implicit `any` types without error; (2) 42 files with `@ts-nocheck` directives that disable the TypeScript compiler entirely for those files; and (3) `skipLibCheck: true` which skips type checking of declaration files. The actual type safety coverage is significantly lower than the zero-error count suggests. Section 2 of this report provides a complete inventory of all suppressed files.

Check	Result	Status
npx tsc --noEmit	0 errors (exit code 0)	PASS*
noImplicitAny	false (weak type safety)	WARN
@ts-nocheck files	42 files suppressing all errors	WARN
@ts-ignore usage	0 files	PASS
@ts-expect-error usage	1 file	INFO

***PASS with caveat:** Zero errors is achieved through suppression, not resolution. True type safety requires removing @ts-nocheck and enabling noImplicitAny.

1.3 Lint Status (npm run lint)

ESLint reports **63 errors and 9 warnings** across the codebase. The lint configuration extends `eslint-config-next` (v16.1.1). The errors are distributed across multiple categories: React hooks violations (calling setState directly within effects), forbidden require-style imports, empty object types, and unused eslint-disable directives. Notably, `9 warnings are auto-fixable` with `--fix`. The errors indicate real code quality issues that should be resolved before Phase 2, particularly the setState-in-effect patterns which can cause cascading render bugs in production.

Category	Count	Severity	Auto-fixable
react-hooks/set-state-in-effect	2	Error	No
@typescript-eslint/no-require-imports	4	Error	No
@typescript-eslint/no-empty-object-type	3	Error	No
Unused eslint-disable directives	4	Warning	Yes
Other	50	Error/Warning	Partial
TOTAL	63 errors, 9 warnings	-	9 auto-fixable

1.4 Prisma Schema Validation (npx prisma validate)

Prisma schema validation **fails** with error code P1012. The schema itself is syntactically valid (67 models, PostgreSQL provider, Prisma 6.19.3 syntax), but the `DATABASE_URL` environment variable is not set in the local environment. This means the Prisma CLI cannot validate the datasource connection string format. In a production deployment (Render.com), this environment variable would be configured. The schema defines 67 models with extensive indexing, cascading deletes, and JSON fields for structured data storage. The failure is expected in the local development environment without a connected database.

Check	Result	Status
npx prisma validate	P1012: DATABASE_URL not set	FAIL (expected)
Schema syntax	Valid - 67 models, PostgreSQL	PASS
Prisma version	6.19.3 (pinned correctly)	PASS
Model count	67 models defined	INFO

> **Note:** This failure is expected. Prisma requires a valid DATABASE_URL to validate. The schema itself is well-formed.

1.5 End-to-End Functionality Assessment

The application **cannot be fully tested end-to-end** in the current environment because: (a) no PostgreSQL database is connected, so all API routes that query the database will return 500 errors at runtime; (b) no email service is configured, so OTP delivery and email sending features are non-functional; (c) no middleware.ts exists, so there is no authentication guard on protected routes. The build compiles successfully, meaning all pages render and API routes are registered. However, the application is in a "compiles but cannot run" state without external service dependencies.

Capability	Status	Blocker
Local dev server	Starts, but DB-dependent routes 500	No DATABASE_URL
Authentication	OTP logic exists, email not configured	No EMAIL_API_KEY
Database queries	Schema valid, cannot execute	No PostgreSQL
Deployment	Build passes, deploy to Render untested	Manual verification needed
Manual testing	Not performed	All of the above

2. @ts-nocheck File Analysis

A total of **42 files** contain `@ts-nocheck` directives, which completely disable TypeScript type checking within those files. This is equivalent to telling the compiler "trust me, I know what I am doing" for approximately 8,400+ lines of code. These files represent every layer of the application: API routes, library modules, UI components, and AI/ML pipeline code. Below is a categorized inventory with disposition decisions for each file.

2.1 API Route Files (22 files)

The majority of @ts-nocheck files are API route handlers. These are the server-side endpoints that power the application. Each one interacts with the Prisma database client and handles HTTP request/response cycles. Type errors in these files are particularly risky because they can cause runtime crashes when database queries return unexpected shapes. All 22 API route files are in active use in the user journey and must be fixed before Phase 2 begins.

File	Lines	Purpose	Journey Impact	Disposition
ai/account-brief/route.ts	259	AI account brief generation	Critical - Revenue Intel	FIX in Phase 2
ai/conversation-plan/route.ts	242	AI conversation plan gen	High - Outreach	FIX in Phase 2
ai/opportunities/route.ts	337	AI opportunity detection	Critical - Pipeline	FIX in Phase 2
ai/query/route.ts	283	Natural language query	High - Dashboard	FIX in Phase 2
ai/score-leads/route.ts	322	Lead scoring engine	Critical - Prioritization	FIX in Phase 2
ai/suggested-contacts/route.ts	220	Contact suggestions	Medium - Research	FIX in Phase 2
analytics/route.ts	295	Analytics aggregation	High - Reports	FIX in Phase 2
batches/route.ts	507	Excel import processing	Critical - Data Import	FIX in Phase 2
companies/[id]/intelligence/route.ts	300	Company intel retrieval	Critical - Company Intel	FIX in Phase 2
contacts/route.ts	111	Contact CRUD	High - Contacts	FIX in Phase 2
drafts/[id]/route.ts	101	Draft email editing	High - Outreach	FIX in Phase 2
emails/send/route.ts	188	Email sending	Critical - Outreach	FIX in Phase 2
export/route.ts	133	Data export to Excel	Medium - Reports	FIX in Phase 2
imports/route.ts	322	CSV/Excel import	Critical - Data Import	FIX in Phase 2
notes/route.ts	220	Note CRUD operations	Medium - Notes	FIX in Phase 2
opportunities/route.ts	90	Opportunity CRUD	High - Pipeline	FIX in Phase 2
opportunities/[id]/route.ts	121	Single opportunity ops	High - Pipeline	FIX in Phase 2
preferences/route.ts	53	User preferences	Medium - Settings	FIX in Phase 2

queue/route.ts	132	Email queue management	High - Outreach	FIX in Phase 2
reports/data-quality/route.ts	205	Data quality metrics	High - Reports	FIX in Phase 2
reports/pipeline/route.ts	151	Pipeline analytics	High - Reports	FIX in Phase 2
reports/revenue/route.ts	107	Revenue analytics	High - Reports	FIX in Phase 2
reports/team-performance/route.ts	136	Team performance	Medium - Reports	FIX in Phase 2
research-agent/route.ts	139	Deep research agent	High - Research	FIX in Phase 2
sequences/[id]/execute/route.ts	98	Sequence execution	High - Sequences	FIX in Phase 2
sequences/[id]/steps/[stepId]/route.ts	123	Sequence step editing	Medium - Sequences	FIX in Phase 2
signals/route.ts	369	Signal management	Critical - Intel	FIX in Phase 2
timeline/route.ts	88	Timeline events	Medium - Activity	FIX in Phase 2
verify-queue/process/route.ts	105	Email verification	Medium - Data Quality	FIX in Phase 2

2.2 Library / AI Pipeline Files (7 files)

These files form the core AI and intelligence pipeline. They implement the reasoning engine, strategy generator, usage tracker, brief enhancer, signal detector, brief generator, and account prioritization engine. Together, these represent approximately 2,800 lines of complex business logic that orchestrates AI model calls, database queries, scoring algorithms, and data transformations. Every one of these files is in active use and directly powers the revenue intelligence, account prioritization, and AI recommendation features. These are the highest-risk files because type errors in AI pipeline code can produce silently incorrect recommendations or miscalculated scores without any visible failure.

File	Lines	Purpose	Disposition
ai-copilot/reasoning-engine.ts	733	Main AI orchestrator	FIX in Phase 2
ai-copilot/strategy-generator.ts	492	Engagement strategy gen	FIX in Phase 2
account-prioritization/engine.ts	496	Account scoring engine	FIX in Phase 2
revenue-intelligence/brief-generator.ts	389	Executive brief generation	FIX in Phase 2
revenue-intelligence/signal-detector.ts	228	Signal classification	FIX in Phase 2
ai-copilot/brief-enhancer.ts	232	AI brief enhancement	FIX in Phase 2
ai-copilot/usage-tracker.ts	223	AI usage/cost tracking	FIX in Phase 2

2.3 CRM Legacy Components (5 files)

Five files in the `src/app/crm/` directory carry @ts-nocheck. These are part of the legacy CRM SPA (App.tsx) that uses inline hardcoded data from `data.ts`. The CRM SPA is a **separate UI system** from the main application shell (page.tsx with SCREEN_MAP). In the strategic reframe, the CRM SPA is **not part of**

the active user journey for the intelligence platform positioning. However, some of these components are still referenced in screen-map.tsx as lazy-loaded alternatives. The recommended disposition is to either delete or de-prioritize these files.

File	Lines	Purpose	Journey Impact	Disposition
crm/Settings.tsx	102	Settings screen	Low - Legacy SPA only	DELETE (Phase 3)
crm/Knowledge.tsx	69	Knowledge library	Low - Legacy SPA only	DELETE (Phase 3)
crm/Tasks.tsx	83	Tasks screen	Low - Legacy SPA only	DELETE (Phase 3)
crm/EmailGen.tsx	82	Email generator	Low - Legacy SPA only	DELETE (Phase 3)
crm/components.tsx	128	Shared UI components	Low - Legacy SPA only	DELETE (Phase 3)

2.4 Intelligence Reasoning Screen (1 file)

The `intelligence-reasoning-screen.tsx` (557 lines) is the UI component for the AI reasoning visualization. This is a critical screen in the intelligence platform positioning and is actively used in the user journey. It must be fixed in Phase 2 to ensure type safety for the complex state management and AI reasoning display logic.

File	Lines	Purpose	Disposition
intelligence-reasoning-screen.tsx	557	AI reasoning visualization	FIX in Phase 2

2.5 Critical Production Functionality Assessment

Of the 42 @ts-nocheck files, **37 are in active use** in the intelligence platform user journey. Only the 5 CRM legacy files are candidates for deletion. This means **88% of suppressed files guard critical production functionality**. Removing @ts-nocheck without fixing the underlying type errors will cause immediate build failures. The recommended approach is incremental: remove @ts-nocheck from one file at a time, fix the revealed errors, verify the build still passes, then proceed to the next file.

Category	Count	In Active Journey	Plan
API Routes	29	Yes - All 29	Fix in Phase 2, one file at a time
AI Pipeline Libraries	7	Yes - All 7	Fix in Phase 2, one file at a time
CRM Legacy Components	5	No - Legacy SPA	Delete in Phase 3
UI Screens	1	Yes	Fix in Phase 2
TOTAL	42	37 (88%)	37 to fix, 5 to delete

3. User Journey Validation

This section documents the complete customer journey from the perspective of a new enterprise customer signing up tomorrow. The analysis is based on code inspection of authentication flows, data import pipelines, API routes, and screen components. Where functionality exists but has not been manually tested, this is noted explicitly.

3.1 Step 1: Company Onboarding

The current authentication system is OTP-based. There is no multi-tenant company creation flow. The system uses a single `User` model with a binary role field (admin/user). There is no `Company` or `Organization` model for tenant isolation. The `Company` model in the schema refers to prospect/target companies in the CRM context, not the customer's own organization.

Onboarding flow (as implemented):

1. User navigates to `/login` - a client-side React component (`login-page.tsx`)
2. User enters email and clicks "Send OTP" - calls `POST /api/auth/request-otp`
3. OTP is generated via `lib/otp.ts` - in dev mode, code is returned in response (no email sent)
4. User enters 6-digit code - calls `POST /api/auth/verify-otp`
5. If user does not exist, they are auto-created in the `User` table
6. A session token is created via `lib/session.ts` and stored in a cookie
7. The `/register` endpoint is a **mock** - it returns a hardcoded demo user

Critical gaps for enterprise deployment:

No multi-tenant isolation: All users share the same database with no company/organization boundary

No password authentication flow: OTP-only login means lost email = lost access

Register is a mock: New user creation happens implicitly via OTP verification, not via a registration form

No RBAC beyond admin/user: The User model has only two roles with no granular permissions

No middleware.ts: There is no authentication guard on API routes. The middleware file was deleted and never restored

3.2 Step 2: Data Upload

Data import is handled through two API endpoints: `POST /api/imports` (322 lines, @ts-nocheck) and `POST /api/batches` (507 lines, @ts-nocheck). The import system supports both CSV and Excel file formats using the `xlsx` library (v0.18.5). The import pipeline includes: file hash validation to prevent duplicate uploads, column mapping from source headers to internal fields, email validation (syntax, disposable domain detection, role-based detection, free provider detection), data normalization (industry names, country codes, company name variants), and duplicate detection across existing records.

Import flow (as implemented in code):

1. User navigates to Import screen (import-screen.tsx, 1,663 lines)
2. File is uploaded via multipart form to `/api/imports` or `/api/batches`
3. XLSX library parses the file into row arrays
4. Column mapping rules (ColumnMappingRule model) auto-detect header patterns
5. Each row is validated against FieldValidationRule configurations
6. Normalization mappings are applied to clean messy values
7. Duplicate detection uses file hash + email matching
8. Validated rows are inserted as Contact records linked to Company records
9. An ImportBatch record tracks the entire operation

> **Note:** The DataUpload/UploadRow models (Phase 1 data intelligence engine) define a more sophisticated pipeline with staged review, but the current import endpoint uses the legacy ImportBatch/Contact flow. This creates two parallel import systems that need unification.

3.3 Step 3: Post-Import User Journey

After data import, the user navigates through the application via the SCREEN_MAP system. The main application shell is `src/app/page.tsx` (762 lines), which renders a sidebar navigation, top bar, and content area. Navigation is state-driven using the Zustand store (`lib/store.ts`). The complete mapped journey is shown below, with implementation status for each screen.

Step	Screen	Route Key	Status	Notes
1	Dashboard	dashboard	Implemented	553 lines, shows stats cards
2	Companies List	companies	Implemented	1,211 lines, full CRUD
3	Company Detail	company-detail	Implemented	1,587 lines, tabs for intel/notes/signals
4	Company Intelligence	signal-intelligence	Implemented	Signal detection and classification
5	AI Recommendations	opportunity-radar	Implemented	Opportunity scoring and display
6	Executive Brief	revenue-intelligence-brief	Implemented	AI-generated account briefs
7	Outreach / Email Gen	email-generation	Implemented	Draft generation with AI
8	Pipeline	pipeline	Implemented	Kanban-style pipeline view
9	Reports	reports	Implemented	Revenue, pipeline, data quality
10	Research Agent	research-agent	Implemented	Deep company research
11	Account Ranking	account-ranking	Implemented	Priority scoring display
12	Import	import	Implemented	1,663 lines, Excel/CSV upload
13	Settings	settings	Implemented	Configuration management

All 13 primary journey screens are implemented and registered in the SCREEN_MAP. The application has an additional 47+ screens registered for secondary features (playbooks, strategy room, knowledge library, sequences, etc.). Each screen is lazy-loaded via React.lazy() to optimize initial bundle size.

4. Database Reality Check

The Prisma schema defines **67 models** across 1,744 lines. However, code analysis reveals a significant gap between what is defined in the schema and what is actually used in the application code. This section categorizes every model by its actual usage across API routes and library modules.

4.1 Model Usage Classification

Models were classified by searching for `db.modelName` references across all TypeScript files in the `src/` directory. A model is considered "Actively Used" if it appears in API route files or core library modules. "Library Only" means it is only referenced in @ts-nocheck library files (revenue-intelligence/, ai-copilot/). "Schema Only" means the model exists in the schema but has zero code references anywhere in the application.

Actively Used Models (16) - Referenced in API routes:

Model	API Refs	Primary Role
Contact	60	Core entity - prospect contacts
Company	54	Core entity - target accounts
Draft	20	Email drafts for outreach
Reply	11	Email reply tracking
Bounce	8	Bounce tracking
Suppression	8	Email suppression list
User	7	Authentication and users
Pursuit	3	Sales pursuit tracking
Segment	2	Lead segmentation
Playbook	2	Sales playbooks
Evidence	1	Intelligence evidence
Session	1	User sessions
AuditLog	1 (lib)	Audit trail
SystemSetting	2 (lib)	System configuration
OtpCode	1 (lib)	OTP verification
CompanySignal	7 (lib)	Buying signals

Library-Only Models (19) - Referenced only in @ts-nocheck library files:

Model	Referenced In	Status
AccountBrief	revenue-intelligence/	Schema + Lib - no API route
AccountScore	revenue-intelligence/	Schema + Lib - no API route
OpportunitySignal	revenue-intelligence/, signal-extraction	Schema + Lib - no API route

OpportunityRecommendation	intelligence-confidence.ts	Schema + Lib - no API route
CompanyIntelligenceHealth	intelligence-confidence.ts	Schema + Lib - no API route
IntelligenceObject	revenue-intelligence/	Schema + Lib - no API route
IntelligenceTimeline	revenue-intelligence/	Schema + Lib - no API route
KnowledgeEntry	revenue-intelligence/, ai-copilot/	Schema + Lib - no API route
Evidence	revenue-intelligence/	Schema + Lib - no API route
CompanyResearchCard	account-prioritization/	Schema + Lib - no API route
SignalCapabilityMatch	account-prioritization/	Schema + Lib - no API route
CompanyNote	API routes	Schema + API - minimal usage
CompanyTimelineEvent	API routes	Schema + API - minimal usage
CompanyAlias	Schema only	No code references found
KnowledgeVersion	Schema only	No code references found
SourceHealth	Schema only	No code references found
HumanIntelligenceInbox	Schema only	No code references found
IntelligenceAlert	Schema only	No code references found
IntelligenceAssociation	Schema only	No code references found

Schema-Only Models (32) - Defined in schema but never referenced in code:

The following 32 models exist in the Prisma schema but have **zero references** anywhere in the application code. These represent planned features, abandoned features, or scaffolding that was defined but never implemented. They occupy database space and schema complexity without providing any functional value.

```
ImportBatch, ContactNote, CapabilityAsset, EmailTemplate, EmailSequence,
SequenceStep, SequenceEnrollment, SendQueue, EmailEvent, ABTest, SegmentContact,
ConversationPlan, AccountStrategy, DataUpload, UploadRow, ColumnMappingRule,
FieldValidationRule, NormalizationMapping, ScoringWeight, NormalizationLog,
DataQualityScore, Job, JobLog, AIGenerationAudit, IntelligenceValidation,
SignalValidation, IntelligenceConflict, PriorityScoreHistory,
RecommendationFeedback, EvidenceSourceReliability, Connector, ConnectorRun
```

4.2 Specific Model Assessment

Task model: Does not exist in the schema. The UI has a tasks-screen.tsx, but it renders an empty placeholder because the underlying data model was never created. This screen is non-functional.

StrategicInsight model: Referenced in the @ts-nocheck ai-copilot/reasoning-engine.ts as `db.strategicInsight`, but this model does not exist in the current schema. This would cause a runtime Prisma error. This is a ghost reference from removed code.

AIUsageLog model: Does not exist in the schema. The ai-copilot/usage-tracker.ts (223 lines, @ts-nocheck) tracks AI usage but appears to use SystemSetting or AIGenerationAudit models instead. This is misleading naming.

Opportunity model: Does not exist as a standalone model. The system uses OpportunityRecommendation (schema) for AI-detected opportunities and a separate "opportunities" API route that likely uses Company/pursuit data. There is a naming confusion between the API route name and the actual schema model.

Timeline model: Two timeline models exist: CompanyTimelineEvent (old system, minimal API usage) and IntelligenceTimeline (new system, library-only). They serve overlapping purposes and need consolidation.

Research models: CompanyResearchCard (schema + library-only), Evidence (schema + library), and IntelligenceObject (schema + library) form the research data layer. All three are accessed only from @ts-nocheck files, meaning their type safety is completely unverified.

5. Functional Testing Status

No functional testing has been performed on this application. The codebase includes a Vitest configuration (v4.1.10) and some test files in `src/lib/revenue-intelligence/__tests__/`, but these tests have not been executed as part of this assessment. The following test plan defines the minimum manual and automated validation required before Phase 2 execution.

5.1 Authentication Test Plan

Test Case	Steps	Expected Result	Priority
New user OTP login	Enter new email, request OTP, verify code	User created, session established	P0
Existing user OTP login	Enter existing email, request OTP, verify	Session established, lastLoginAt updated	P0
Password login	Enter email + password	Password verified, OTP sent for 2FA	P0
Invalid OTP	Enter wrong 6-digit code	401 error, attempts incremented	P0
Expired OTP	Wait for expiry, then verify	401 error with expiry message	P1
Session persistence	Login, close browser, reopen	Session still valid if not expired	P0
Logout	Click logout	Session destroyed, redirected to login	P0
Protected route access	Access /app without session	Redirected to login (requires middleware)	P0
Rate limiting	Request OTP 10 times in 1 minute	429 rate limit response	P1

5.2 Data Import Test Plan

Test Case	Input	Expected Result	Priority
Small Excel import	50 companies, 200 contacts	All rows imported, batch status completed	P0
Large Excel import	2000 companies, 10000 contacts	Batch processing, progress tracking	P0
Duplicate records	Same email in two rows	Second row flagged as duplicate	P0
Invalid emails	test@, missing TLD, spaces	Rows flagged with validation errors	P0
Column mapping	Non-standard headers	Auto-mapping via regex rules	P1
Empty file	XLSX with headers only	Zero rows accepted, batch completed	P1
Mixed data types	Numbers in text fields	Normalization applied	P2
Concurrent imports	Two files uploaded simultaneously	No data corruption, both batches tracked	P1

5.3 AI Features Test Plan

Test Case	Steps	Expected Result	Priority
Company enrichment	Trigger enrichment for a company	ResearchCard populated, freshness updated	P0
Lead scoring	Run scoring on imported contacts	leadScore computed, ranking updated	P0
AI recommendations	View opportunity radar	OpportunityRecommendations generated	P0
Executive brief	Generate brief for target company	AccountBrief created with AI content	P0
Conversation plan	Generate plan for executive	ConversationPlan saved	P1
Research agent	Run deep research query	IntelligenceObjects created from research	P1

5.4 CRM and Outreach Test Plan

Test Case	Steps	Expected Result	Priority
Company creation	Create company via API/UI	Company record created with all fields	P0
Contact management	Add/edit/delete contacts	CRUD operations succeed	P0
Pipeline stages	Move deal through pipeline	Stage transitions recorded	P1
Email generation	Generate draft for contact	Draft created with AI content	P0
Email sending	Send draft via queue	SendQueue updated, provider ID stored	P1
Reply tracking	Simulate reply webhook	Reply record created, status updated	P1
Multi-company isolation	Data from different batches	No cross-contamination (not yet supported)	P0

6. UI/UX Assessment

The application has **62 screen components** registered in `screen-map.tsx`, plus a separate legacy CRM SPA with 11 components. The main application uses a sidebar navigation pattern with lazy-loaded screen components, framer-motion page transitions, and a shadcn/ui component library. No screenshots or videos have been captured for this assessment. The analysis is based on code inspection of screen components.

6.1 Screen Inventory

The application offers an extensive set of screens covering the full intelligence platform workflow. Screens are categorized into functional groups below. All screens are registered in the SCREEN_MAP and accessible via the sidebar navigation.

Group	Screens	Count
Core CRM	Dashboard, Companies, Company Detail, Contacts, Contact Detail, Pipeline	6
Intelligence	Command Center, Signal Intelligence, Research Agent, Mind Map	4
Revenue Intel	Revenue Intel, Brief, Opportunities, Recommendations, Account Ranking	5
AI Pipeline	Intelligence Reasoning, AI Strategy, Opportunity Radar, Conversation Studio	4
Outreach	Email Generation, Drafts, Queue, Sequences, Templates	5
Data Management	Import, Leads, Segments, Duplicates, Data Health	5
Reporting	Reports, Analytics, Audit, Data Quality	4
Settings	Settings, ICP Settings, Prompt Templates, Knowledge Library	4
Operations	Tasks, Playbooks, Strategy Room, Pursuit Workspace	4
Comms	Replies, Bounces, Relationship Memory	3
Advanced	Opportunity Workspace, Intelligence Health, Sources, Knowledge, Timeline, Inbox	6+
Legacy CRM SPA	Dashboard, Companies, Contacts, Opportunities, Tasks, Settings, Knowledge, Email	11 (separate)

6.2 Product Maturity Classification

Based on code inspection, the application currently falls between **Category A (Internal Developer Tool)** and **Category B (Enterprise Software Product)**. It has the feature breadth of an enterprise product but lacks the polish, type safety, error handling, and production hardening expected of a commercial product. The investor feedback that it looks like a "high school weekend project" likely reflects several observable issues: inconsistent UI patterns across 62 screens, absence of loading states and empty states in many screens, no toast notification system for user feedback, hardcoded demo data remnants, and the general "wide but shallow" feeling of having many screens with limited depth in each.

Attribute	Current State	Enterprise Standard	Gap

Type Safety	42 @ts-nocheck files, noImplicitAny=false	Zero @ts-nocheck, strict=true	Critical
Error Handling	Screen-level error boundaries	Global error handling + toast + retry	High
Loading States	Skeleton loader on main page only	Per-screen loading skeletons	High
Empty States	Not implemented in most screens	Helpful empty state with CTA	Medium
Multi-tenancy	Not implemented	Company/organization isolation	Critical
Authentication	OTP only, no middleware	MFA + middleware + RBAC	Critical
Testing	Vitest config exists, few unit tests	Integration + E2E test suite	High
Documentation	Code comments only	API docs + user guide + runbook	Medium

Recommendation: Before any investor demo, the application must pass the "screenshot test" - every screen must look polished with proper loading states, empty states, and consistent spacing. The current 62-screen scope is too wide for the team's capacity. A focused demo flow covering 5-8 key screens with exceptional polish will outperform a broad but shallow tour of 62 mediocre screens.

7. Phase 2 Execution Plan

Phase 2 focuses on the most critical infrastructure gap: **restoring middleware.ts with authentication, security headers, rate limiting, and CSRF protection**. Without middleware, all API routes are publicly accessible with no authentication guard. This is the single most important security fix and must be completed before any other Phase 2 work.

7.1 Phase 2 Objectives

Objective	Description	Priority
Restore middleware.ts	Create src/middleware.ts with authentication guard for all /api/ routes	P0 - Critical
Session validation	Verify session token on every API request, return 401 if invalid	P0 - Critical
Security headers	Add X-Content-Type-Options, X-Frame-Options, CSP headers via middleware	P0 - Critical
Rate limiting	Implement IP-based rate limiting for auth endpoints (OTP, login)	P1 - High
CSRF protection	Add CSRF token validation for state-changing requests	P1 - High
Admin route protection	Restrict /api/seed, /api/reset to admin-only access	P1 - High

7.2 Files Affected

File	Action	Risk
src/middleware.ts	CREATE - new file	Medium - must not block static assets
src/lib/session.ts	MODIFY - add validateSession export	Low - existing code
src/lib/auth-helpers.ts	CREATE - helper functions for middleware	Low - new file
next.config.ts	MODIFY - remove header() if middleware handles it	Low - currently has headers()
All /api/auth/* routes	VERIFY - ensure they are excluded from auth guard	Medium - OTP routes must be public
src/app/api/seed/route.ts	VERIFY - admin-only protection	Low
src/app/api/reset/route.ts	VERIFY - admin-only protection	Low

7.3 Expected Risks

Risk	Probability	Impact	Mitigation
Static assets blocked	Medium	High - site breaks	Exclude /_next/static, /favicon.ico from middleware
Login page blocked	Medium	Critical - cannot login	Exclude /login, /signup, /api/auth/* from auth guard

Session cookie mismatch	Low	High - all requests fail	Test thoroughly with dev OTP flow
Performance regression	Low	Medium - slower requests	Use lightweight session check (DB lookup or JWT)
WebSocket connection blocked	Low	Medium - real-time breaks	Exclude /api/realtime from middleware

7.4 Acceptance Criteria

Phase 2 is complete when all of the following conditions are met:

- `src/middleware.ts` exists and contains authentication logic
- Accessing any `/api/*` route without a valid session returns HTTP 401
- `/api/auth/*` routes (login, OTP, register) remain publicly accessible
- Static assets (`/_next/*`, images, fonts) are not affected by middleware
- Security headers (X-Content-Type-Options, X-Frame-Options, CSP) are present on API responses
- Rate limiting prevents more than 5 OTP requests per email per minute
- `npm run build` still passes with `ignoreBuildErrors: false`
- `npx tsc --noEmit` still passes (no new errors introduced)
- Manual test: login flow works end-to-end with middleware active
- Manual test: unauthenticated access to dashboard API returns 401

Estimated effort: 4-6 hours for middleware.ts creation, testing, and verification. The main complexity is in getting the exclusion rules correct so that public routes (login, OTP, static assets) are not accidentally blocked.